LABORATORY 6: TIVA – RPI Integration

Part 1: UART.

Theory Concepts:

UART is an asynchronous serial communication protocol that enables communication between 2 devices. This communication protocol sends 1 byte per time sequentially, for that reason Uart uses different speed for the communication, generally we use 9600 bauds.

Wires	2
Speed	9600, 19200, 38400, 57600, 115200, 230400, 460800, 921600, 1000000, 1500000
Methods of Transmission	Asynchronous
Maximum Number of Masters	1
Maximum Number of Slaves	1

UART uses a packet where a start bit is sent, then the data frame, then the parity bit (if used) and finally the stop bit.

How does the start, parity and stop bit work? Explain it with an example.

UART in TIVA:

To use UART in TIVA we will need to:

- Import libraries.
- Enable UART ports and peripherals.
- Configure UART baud rate.
- Send and receive data.

Import libraries:

```
//Libraries needed for TIVA funct
#include <stdint.h>
#include <stdbool.h>
#include "inc/hw_memmap.h"
#include "driverlib/debug.h"
#include "driverlib/gpio.h"
#include "driverlib/sysctl.h"
#include "driverlib/uart.h"
#include "driverlib/ethernet.h"
#include "driverlib/interrupt.h"
#include "driverlib/pin_map.h"
#include "utils/uartstdio.c"
```

Enable UART:

```
//Enable the UART periph
SysCtlPeripheralEnable(SYSCTL_PERIPH_UART0);
SysCtlPeripheralEnable(SYSCTL_PERIPH_TIMER0);
SysCtlPeripheralEnable(SYSCTL_PERIPH_GPIOA);
//UART module 0 receive//
GPIOPinConfigure(GPIO_PA0_U0RX);
//UART module 0 transmit//
GPIOPinConfigure(GPIO_PA1_U0TX);
//Define UART pins
GPIOPinTypeUART(GPIO_PORTA_BASE, 0x03);
```

The previous lines go inside the main function. Is necessary to enable the peripheral, configure the pin for transmitter TX and receiver RX and define the pins for UART.

UART Baud Rate:

```
//Baudrate de UART//(Base periph, clock freq, baud rate)
UARTStdioConfig(0,9600,120000000);
```

UART use:

```
while(1)
{
 UARTprintf(msg);
 UARTgets(data,100);
 strcat(data,"\n");
 UARTprintf(data);
}
```

UARTprintf() sends data

UARTgets receives data

We can use terminal to determine the port where TIVA is connected

```
c1294xl/pruebitaUART: sudo ls /dev/tty
tty        tty21      tty36      tty5       tty62      ttyS16     ttyS3
tty0       tty22      tty37      tty50      tty63      ttyS17     ttyS30
tty1       tty23      tty38      tty51      tty7       ttyS18     ttyS31
tty10      tty24      tty39      tty52      tty8       ttyS19     ttyS4
tty11      tty25      tty4       tty53      tty9       ttyS2      ttyS5
tty12      tty26      tty40      tty54      ttyACM0    ttyS20     ttyS6
tty13      tty27      tty41      tty55      ttyprintk  ttyS21     ttyS7
tty14      tty28      tty42      tty56      ttyS0      ttyS22     ttyS8
tty15      tty29      tty43      tty57      ttyS1      ttyS23     ttyS9
tty16      tty3       tty44      tty58      ttyS10     ttyS24
tty17      tty30      tty45      tty59      ttyS11     ttyS25
tty18      tty31      tty46      tty6       ttyS12     ttyS26
tty19      tty32      tty47      tty60      ttyS13     ttyS27
tty2       tty33      tty48      tty61      ttyS14     ttyS28
tty20      tty34                            ttyS15     ttyS29
```

UART in Rasp:

For the raspberry, the UART usage is easier than TIVA, we must remember to select the port where our TIVA is connected. Another good practice is to manage all the UART operations using a try/except block.

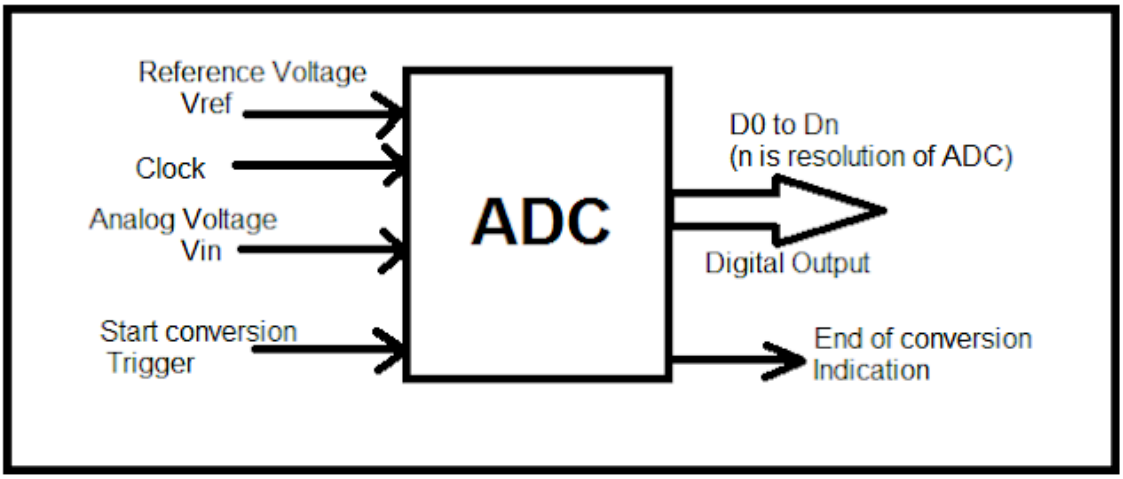
What is try/except? How do you use it in Python? Which advantages and disadvantages it has?

```
import serial
from time import sleep

ser = serial.Serial("/dev/ttyACM0", 9600) #Se inicia la comunicacion serial con
#los parametros de: nombre del dispositivo serial, baud rate, tiempo para leer operaciones
ser.reset_input_buffer()#limpia cualquier byte innecesario

while True:
    try:
        if ser.in_waiting > 0: #se revisda si algun dato esta disponible, si es asi se procede a leer el dato
            value = ser.readline().decode('utf-8').rstrip()
            print(value)
    except Exception as e:
        print(e)
```

Part 2: ADC.

Theory Concepts:

Analog to Digital Converter or ADC is a module that allows us to convert analog signals into digital ones. This is important when we work with embedded systems because most of the real-world signals will be interpreted as analog signals.



UART uses a packet where a start bit is sent, then the data frame, then the parity bit (if used) and finally the stop bit.

How does the start, parity and stop bit work? Explain it with an example.

ADC in TIVA:

To use ADC in TIVA we will need to:

- Import libraries.
- Enable ADC ports and peripherals.

- Select ADC channel.
- Capture converted value.

Import libraries:

```
//Potenciometro mediante ADC//
#include <stdint.h>
#include <stdbool.h>
#include "inc/tm4c1294ncpdt.h"
#include "inc/hw_memmap.h"
#include "driverlib/sysctl.h"
#include "driverlib/gpio.h"
#include "driverlib/timer.h"
#include "driverlib/interrupt.h"
#include "driverlib/adc.h"
```

Enable ADC:

```
// Enable the ADC0 module.
SysCtlPeripheralEnable(SYSCTL_PERIPH_ADC0);
// Enable Port and pin for ADC//
SysCtlPeripheralEnable(SYSCTL_PERIPH_GPIOK);
//User Led pin declaration
GPIOPinTypeGPIOOutput(GPIO_PORTN_BASE, 0x03);
GPIOPinTypeGPIOOutput(GPIO_PORTF_BASE, 0x11);
//Enables pin
GPIOPinTypeADC(GPIO_PORTK_BASE, 0x08);
```

The previous lines go inside the main function. We must enable the adc peripheral and define the pin type as adc. Which pins are we defining?

ADC configuration:

```
// Enable the first sample sequencer to capture the value of channel 0 when
// the processor trigger occurs. Sequencer 3 captures 1 sample
ADCSequenceConfigure(ADC0_BASE, 3, ADC_TRIGGER_PROCESSOR, 0);
//Configures the ADC step (interruption, end and channel 19)
ADCSequenceStepConfigure(ADC0_BASE, 3, 0, ADC_CTL_IE | ADC_CTL_END | ADC_CTL_CH19);
//Enables ADC Sequence
ADCSequenceEnable(ADC0_BASE, 3);
//Clears the ADC interruption
ADCIntClear(ADC0_BASE, 3);
```

Why do we use the channel 19? To select the channel, we must check the TM4C1294XL datasheet in the ADC section:

AIN12	4	PD3	I	Analog	Analog-to-digital converter input 12.
AIN13	3	PD2	I	Analog	Analog-to-digital converter input 13.
AIN14	2	PD1	I	Analog	Analog-to-digital converter input 14.
AIN15	1	PD0	I	Analog	Analog-to-digital converter input 15.
AIN16	18	PK0	I	Analog	Analog-to-digital converter input 16.
AIN17	19	PK1	I	Analog	Analog-to-digital converter input 17.
AIN18	20	PK2	I	Analog	Analog-to-digital converter input 18.
AIN19	21	PK3	I	Analog	Analog-to-digital converter input 19.

ADC use:

```
while(1){
    // Trigger the sample sequence.
    ADCProcessorTrigger(ADC0_BASE, 3);
    // Wait until the sample sequence has completed.
    while(!ADCIntStatus(ADC0_BASE, 3, false))
    {
    }
    //Clear the ADC interruption
    ADCIntClear(ADC0_BASE, 3);
    // Read the value from the ADC.
    ADCSequenceDataGet(ADC0_BASE, 3, &ui32Value);
    //Operates with the ADC values
    if (ui32Value > 2047)
    {
        GPIOPinWrite(GPIO_PORTN_BASE,0x03,0x03);
        GPIOPinWrite(GPIO_PORTF_BASE,0x11,0x11);
    }else{
        GPIOPinWrite(GPIO_PORTN_BASE,0x03,0x00);
        GPIOPinWrite(GPIO_PORTF_BASE,0x11,0x00);
    }
}
```

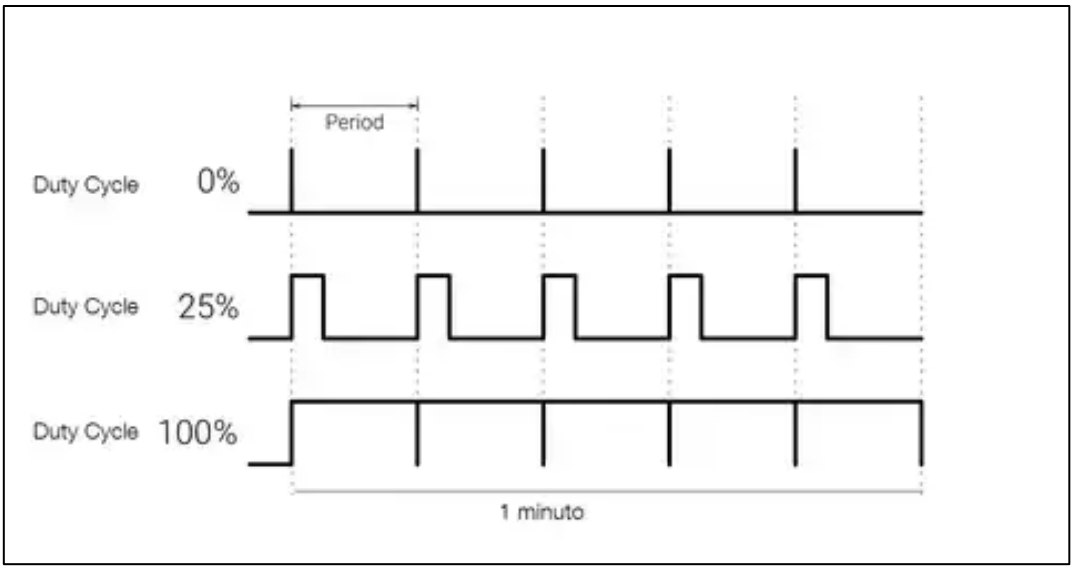
To use adc we need to know the TIVA resolution. With that resolution we can capture the converted value and use it.

- Which is the TIVA resolution?
- Can you give 3 examples of ADC use?

Part 3: PWM.

Theory concepts:

PWM or pulse width modulation is a technique that help us to simulate analog values as output. This is very useful because it allows us to modify the standard behavior from different electronic components such as motors, LEDs, buzzers, etc.



PWM in TIVA:

To use pwm in tiva we must:

- Import pwm libraries.
- Enable peripherals.
- Define the clock tick radius.
- Implement pwm.

To import the pwm library we need:

```
//PWM//
#include <stdint.h>
#include <stdbool.h>
#include "inc/hw_ints.h"
#include "inc/hw_memmap.h"
#include "inc/hw_types.h"
#include "driverlib/debug.h"
#include "driverlib/gpio.h"
#include "driverlib/pin_map.h"
#include "driverlib/sysctl.h"
#include "driverlib/pwm.h"

uint32_t reloj;
volatile uint32_t width;
```

Then we must enable the peripherals to work with pwm:

```
// Enable the PWM0 peripheral
SysCtlPeripheralEnable(SYSCTL_PERIPH_PWM0);
SysCtlPeripheralEnable(SYSCTL_PERIPH_GPIOF);

GPIOPinTypeGPIOOutput(GPIO_PORTF_BASE, GPIO_PIN_2);
// Configure the PWM function for this pin.
GPIOPinConfigure(GPIO_PF1_M0PWM1);
GPIOPinTypePWM(GPIO_PORTF_BASE, GPIO_PIN_1);
// Wait for the PWM0 module to be ready.
while(!SysCtlPeripheralReady(SYSCTL_PERIPH_PWM0))
{
}
```

Similarly, to ADC, we need to check TIVA datasheet PWM pins. We will find a table like this:

Pin Name	Pin Number	Pin Mux / Pin Assignment	Pin Type	Buffer Type	Description
M0FAULT0	46	PF4 (6)	I	TTL	Motion Control Module 0 PWM Fault 0.
M0FAULT1	61	PK6 (6)	I	TTL	Motion Control Module 0 PWM Fault 1.
M0FAULT2	60	PK7 (6)	I	TTL	Motion Control Module 0 PWM Fault 2.
M0FAULT3	81	PL0 (6)	I	TTL	Motion Control Module 0 PWM Fault 3.
M0PWM0	42	PF0 (6)	O	TTL	Motion Control Module 0 PWM 0. This signal is controlled by Module 0 PWM Generator 0.
M0PWM1	43	PF1 (6)	O	TTL	Motion Control Module 0 PWM 1. This signal is controlled by Module 0 PWM Generator 0.
M0PWM2	44	PF2 (6)	O	TTL	Motion Control Module 0 PWM 2. This signal is controlled by Module 0 PWM Generator 1.

Is needed to configure PWM in TIVA, for that we have to calculate the clock tick calculus, we can use the following formula:

Clock tick calculus = target period * clock freq

```
// Configure the PWM generator for count down mode with immediate updates to the parameters.
PWMGenConfigure(PWM0_BASE, PWM_GEN_0, PWM_GEN_MODE_DOWN | PWM_GEN_MODE_NO_SYNC);
// Set the period. For a 50 KHz frequency, the period = 1/50,000, or 20 microseconds.
//For a 20 MHz clock, this translates to 400 clock ticks. Use this value to set the period.
PWMGenPeriodSet(PWM0_BASE, PWM_GEN_0, 400);
// Set the pulse width of PWM1 for a 75% duty cycle(width = 300).
width = 300;
PWMPulseWidthSet(PWM0_BASE, PWM_OUT_1, width);
// Start the timers in generator 0.
PWMGenEnable(PWM0_BASE, PWM_GEN_0);
// Enable the outputs.
PWMOutputState(PWM0_BASE, (PWM_OUT_1_BIT), true);
```

In the example we calculate: 20MHz * 20u sec

Finally, is important to manipulate the clock tick as our pulse width where the maximum pulse will be the total value of the clock tick.

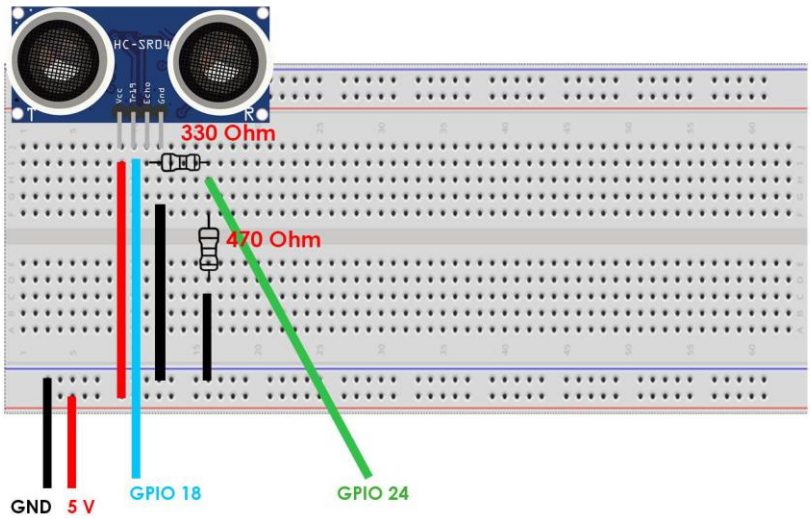
```
while(1){
    for(width=0;width<=399;width++){
        GPIOPinWrite(GPIO_PORTF_BASE,GPIO_PIN_2, GPIO_PIN_2);
        PWMPulseWidthSet(PWM0_BASE, PWM_OUT_1, width);
        SysCtlDelay(9000000);
    }
}
```

How can we use PWM in raspberry pi? Explain the steps.

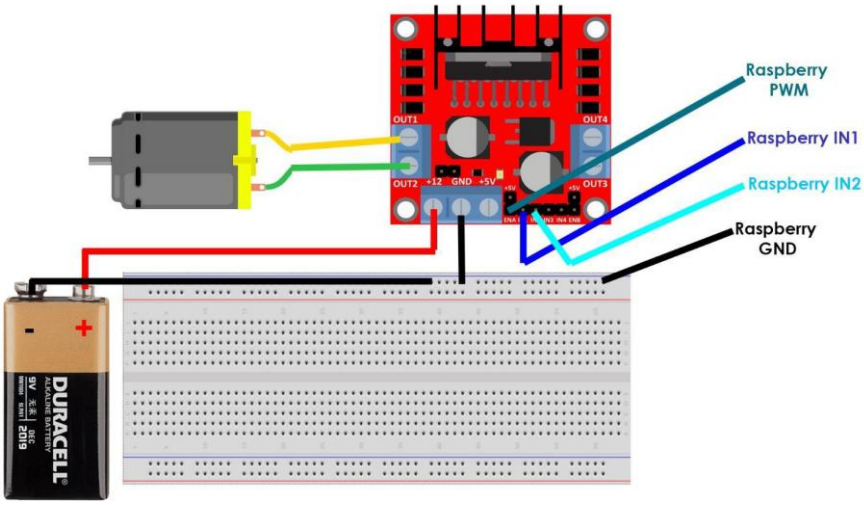
Exercises:

- **Connect a potentiometer to TIVA and make it vary the FS from a custom sequence using timer 0.**
- **Detect the distance of an object using the HC-SR04 ultrasonic sensor in Raspberry Pi. Then send the value of the distance to the Tiva with serial communication using UART. Finally follow the next instructions:**
    - **If the value of distance is above 10 cm, turn of all the user LEDs in the TIVA.**
    - **If the value of distance is between 10 – 8 cm, turn on the user LED PN1.**
    - **If the value of distance is between 8 – 6 cm, turn on the user LEDs PN1 and PN0.**
    - **If the value of distance is between 6 – 4 cm, turn on the user LEDs PN1, PN0 and PF4.**
    - **If the value of distance is below 4 cm, turn on all the user LEDs in the TIVA.**

You can use the following diagram to connect HC-SR04:



- **Using Pulse Width Modulation (PWM) change the speed of a DC motor following Figure 1. For this purpose, set the duty cycle to 75% and test the motor. Repeat this process with the following duty cycles: 25%, 45% and 50%. You can use the following schema for motor connection to raspberry Pi.**



- **Create a sequence using PWM to speed up a DC motor. The duty cycle of the motor must be initiated at 0% and every half of a second the value must increase by 1%. When the duty cycle is at his maximum, the value should return to 0.**
- **Using a potentiometer generate a PWM output and increase the intensity of an external LED (do not use any of the user LEDs) every half of a second. Finally, change the LED for a DC motor.**